`wait()` (because you've just added an item to the `data_queue`). There's no guarantee which thread will be notified or even if there's a thread waiting to be notified; all the processing threads might be still processing data.

Another possibility is that several threads are waiting for the same event, and all of them need to respond. This can happen where shared data is being initialized, and the processing threads can all use the same data but need to wait for it to be initialized (although there are better mechanisms for this; see section 3.3.1 in chapter 3), or where the threads need to wait for an update to shared data, such as a periodic reinitialization. In these cases, the thread preparing the data can call the `notify_all()` member function on the condition variable rather than `notify_one()`. As the name suggests, this causes *all* the threads currently executing `wait()` to check the condition they're waiting for.

If the waiting thread is going to wait only once, so when the condition is `true` it will never wait on this condition variable again, a condition variable might not be the best choice of synchronization mechanisms. This is especially true if the condition being waited for is the availability of a particular piece of data. In this scenario, a *future* might be more appropriate.

4.2. Waiting for one-off events with futures

Suppose you?re going on vacation abroad by plane. Once you get to the airport and clear the various check-in procedures, you still have to wait for notification that your flight is ready for boarding, possibly for several hours. Yes, you might be able to find some means of passing the time, such as reading a book, surfing the internet, or eating in an overpriced airport café, but fundamentally you?re just waiting for one thing: the signal that it?s time to get on the plane. Not only that, but a given flight goes only once; the next time you?re going on vacation, you?ll be waiting for a different flight.

The C++ Standard Library models this sort of one-off event with something called a *future*. If a thread needs to wait for a specific one-off event, it somehow obtains a future representing this event. The thread can then periodically wait on the future for short periods of time to see if the event has occurred (check the departures board) while performing some other task (eating in the overpriced café) in between checks. Alternatively, it can do another task until it needs the event to have happened before it can proceed and then just wait for the future to become *ready*. A future may have data associated with it (such as which gate your flight is boarding at), or it may not. Once an event has happened (and the future has become *ready*), the future can't be reset.

There are two sorts of futures in the C++ Standard Library, implemented as two class templates declared in the `<future>` library header: *unique futures* (`std::future<>`) and *shared futures* (`std::shared_future<>`). These are modeled after `std::unique_ptr` and `std::shared_ptr`. An instance of `std::future` is the one and only instance that refers to its associated event, whereas multiple instances of `std::shared_future` may refer to the same event. In the latter case, all the instances will become *ready* at the same time, and they may all access any data associated with the event. This associated data is the reason these are templates; just like `std::unique_ptr` and `std::shared_ptr`, the template parameter is the type of the associated data. The `std:future<void>, std::shared_future<void>` template specializations should be used where there's no associated data. Although futures are used to